ВСТУП

Розв’язання багатьох задач потребує використання і обробки текстових даних. Ці дані використовуються в найпростішому випадку в разі необхідності виведення різноманітних підказок для забезпечення більш-менш комфортного введення даних. Але існує безліч задач, які безпосередньо орієнтовані на введення, обробку і виведення текстової інформації.

Мета роботи – освоєння методики подання і обробки символьних даних і текстових рядків.

1. ТЕОРЕТИЧНІ ОСНОВИ

1.1. Загальні положення

Для подання текстової інформації у мові C++ використовуються символи (константи) і символьні змінні.

Базовим типом для символьних даних є тип **`char`**. Опис символьних змінних має такий вигляд:

```
char список_імен_змінних;
```

Наприклад:

```
char a, z;
```

Символьні константи позначаються символом, розташованим між двома апострофами (наприклад, `'V'`). Символи з кодами від 0 до 31 є керуючими і їх не можна ввести з клавіатури. У мові C для них уведені керуючі символьні константи, що задаються комбінацією символу \ і деякого іншого символу. Ця комбінація укладається в апострофи при задаванні відповідної символьної константи. Наявні такі керуючі символи:

`'\b'` – повернення на один символ (BS, «забій»);

`'\f'` – переведення сторінки;

`'\n'` – переведення рядка;

`'\r'` – повернення каретки;

`'\t'` – горизонтальна табуляція;

`'\v'` – вертикальна табуляція;

`'\"'` – подвійна лапка;

`'\''` – апостроф;

`'\\'` – зворотна коса риска;

`'\0'` – нульовий символ;

`'\a'` – звуковий сигнал;

`'\?'` – знак питання.

Для запису символу можуть також використовуватися так звані ескейп-послідовності, окремим випадком яких є послідовності, що містять вісімкові та шістнадцяткові коди символів. Для цього після зворотної косої указується вісімковий код (вісімкове число від 0 до 377) або символ `x` чи `X` та шістнадцятковий (шістнадцяткове число від `0` до `FF` включно). Наприклад, символ `'j'` може бути представлений так:

```
'\152'    '\x6A'    '\X6A'    '\x6a'    '\x6A'
```

Одним із варіантів подання рядків є варіант, що прийшов з мови С, де для подання рядкових даних використовується не окремий тип, а масиви символів, причому рядком вважається вміст символьного масиву до першого символу `'\0'`.

1.2. Уведення і виведення символьних даних

Для введення і виведення символьних значень у форматних рядках бібліотечних функцій `printf()` і `scanf()` використовується специфікація перетворення `%c`. Указані функції є небезпечними. Рекомендується замість них використовувати функції `printf_s()` і `scanf_s()`. Функції повертають кількість виведених (уведених) символів або від'ємне значення при помилці.

Особливістю функцій `printf_s()` і `scanf_s()` є те, що вони здійснюють контроль того, що вміст форматного рядка є допустимим. При використанні функції `printf_s()` і `scanf_s()` для виведення рядків або символів, є те що одразу за значенням, що виводиться, у списку виведення необхідно обов'язково вказувати розмір буфера, вмістя якого виводиметься.

Крім перелічених вище функцій, для введення і виведення символів у бібліотеці передбачені спеціальні функції обміну:

`int` `getchar(`**`void`**`);` – функція без параметрів. Дозволяє читати із вхідного потоку (зазвичай із клавіатури) по одному символу за звернення. Як і при використанні функції `scanf()`, зчитування даних починається після натискання клавіші <Enter>. Це дає можливість виправляти інформацію, що вводиться, стираючи останні введені символи за допомогою клавіші <Backspace> до початку її читання програмою;

`int` `putchar(`**`int`** `Ch);` – виводить символьне значення `Ch` у стандартний вихідний потік (зазвичай на екран дисплея);

`int` `getch (`**`void`**`);` – зчитує один символ із клавіатури без відображення на екрані;

int getche (**void**); – зчитує один символ із клавіатури з відображенням його на екрані;

int putch (**int** Ch); – записує символ Ch до стандартного потоку виведення.

Функції getch(), getche() і putch() є застарілими. Замість них стандарт ISO C++ рекомендує використовувати функції

```
int _getch(void);
int _getche(void);
int _putch(int Ch);
```

Можливе використання і таких функцій:

```
int _getchar_nolock(void);
int _getch_nolock(void);
int _getche_nolock(void);
int _putchar_nolock(int Ch);
int _putch_nolock(int Ch);
```

Однак використання цих функцій є безпечним тільки в однопотічних програмах.

Функції printf(), scanf(), getchar(), putchar() і всі їх модифікації описані в заголовному файлі stdio.h, а всі інші перелічені вище функції – у файлі conio.h.

У мові C++ символьні дані, як і дані інших типів, можуть вводитися за допомогою об'єкту cin, а виводитися – за допомогою cout:

```
char ch1, ch2;
cin >> ch1 >> ch2;
cout << ch1 << ch2;
```

Слід ураховувати, що при введенні символу за допомогою cin змінні символьного типу інтерпретуються як би двояко. Спочатку така змінна інтерпретується як цілочислова, наслідком чого є те, що всі пробільні символи на початку рядка введення пропускаються із сприйманням першого символу з кодами більшими коду пробілу, після чого зчитується один символ, який записується у відповідну змінну, уже як символьну. Якщо далі здійснюється введення ще однієї символьної змінної, то процес повториться. При цьому власне введення проводиться тільки після натискання клавіші <Enter>. Символ же, який генерується цією клавішею (символ '\n'), залишається в буфері клавіатури і буде оброблятися наступними операторами введення. Якщо після введення символу (як і після введення числа) здійс-

нюється введення текстового рядка (див. далі), то буде введено порожній рядок.

1.3. Деякі функції обробки символьних даних

int isalnum(**int** Ch); – результат – не нуль, якщо Ch є кодом латинської літери (A, …, Z, a, …, z) або цифри (0, …, 9), і нуль – у противному випадку;

int isalpha(**int** Ch); – результат – не нуль, якщо Ch є кодом латинської літери (A, …, Z, a, …, z), і нуль – у противному випадку;

int isascii(**int** Ch); – результат – не нуль, якщо Ch є кодом ASCII, тобто має значення від 0 до 127 включно, і нуль – у противному випадку;

int iscntrl(**int** Ch); – результат – не нуль, якщо Ch є керуючим символом (має код 0x00, …, 0x01F або 0x7F), і нуль – у противному випадку;

int isdigit(**int** Ch); – результат – не нуль, якщо Ch – цифра у коді ASCII, і нуль – у противному випадку;

int isgraph(**int** Ch); – результат – не нуль, якщо Ch – видимий символ (0x21, …, 0x7E), і нуль – у противному випадку;

int islower(**int** Ch); – результат – не нуль, якщо Ch – є кодом малої латинської літери (a, …, z), і нуль – у противному випадку;

int isprint(**int** Ch); – результат – не нуль, якщо Ch – символ, що може бути надрукований (0x20, …, 0x7E), і нуль – у противному випадку;

int ispunct(**int** Ch); – результат – не нуль, якщо Ch – символ-роздільник (тобто відповідає iscntrl або isspace), і нуль – у противному випадку;

int isspace(**int** Ch); – результат – не нуль, якщо Ch – узагальнений пробіл (пробіл, символ табуляції, символ нового рядка або нової сторінки, символ повернення каретки) з кодами 0x09, …, 0x0D, 0x20, і нуль – у противному випадку;

int isupper(**int** Ch); – результат – не нуль, якщо Ch – є кодом великої латинської літери (A, …, Z), і нуль – у противному випадку;

int isxdigit(**int** Ch); – результат – не нуль, якщо Ch – код шістнадцяткової цифри (0, …, 9, A, …, F, A, a, …, f), і нуль – у противному випадку;

int toascii(**int** Ch); – перетворює ціле число Ch у символ коду ASCII; результатом є число від 0 до 127 включно;

int tolower(**int** Ch); – перетворює код літери Ch до нижнього регістру; коди, що не відповідають латинським літерам, не змінюються;

int toupper(**int** Ch); – перетворює код літери Ch до верхнього регістру; коди, що не відповідають латинським літерам, не змінюються.

Функції isascii() і toascii() є застарілими. Замість них стандарт ISO C++ рекомендує використовувати функції

```
int __isascii(int Ch);
int __toascii(int Ch);
```

Усі перелічені вище функції описані в заголовному файлі ctype.h.

1.4. Рядкові константи

Рядкові константи представляються послідовністю символів, укладеною в лапки (але не в апострофи). Серед символів рядка можуть бути ескейп-послідовності, що відповідають кодам символів (у тому числі й тих, що не мають екранного зображення).

Приклади рядкових констант:

```
"Visual C++"
"Name\tValue\n"
```

Розміщуючи рядкову константу в пам'яті, компілятор автоматично додає в її кінець символ '\0', яким позначається ознака кінця текстового рядка. Рядок може складатися і з одного символу (наприклад, "V"). Проте, у пам'яті він буде займати при цьому два байти (з урахуванням символу, що позначає кінець рядка). У той же час символьна константа (наприклад, 'V') займає один байт.

Можливе використання константи, яка позначає порожній рядок. Для її позначення використовується пара лапок (""), що стоять поруч. Така константа займає в пам'яті 1 байт, використовуваний для зберігання ознаки кінця рядка. Не слід плутати її з рядковою константою, записуваною парою лапок із пробілом між ними (" "), як не слід плутати константи " " і ' '. Два поруч розташованих апострофа ('') діагностуються компілятором як помилкова конструкція, що цілком зрозуміло, оскільки між апострофами повинен бути вказаний символ.

Найпростішою функцією, що служить для обробки рядків, є описана в заголовному файлі string.h функція strlen(), яка обчислює довжину рядка, що задається її параметром:

```
unsigned strlen(const char *Str);
```

1.5. Уведення і виведення символьних рядків

Для введення і виведення рядкових даних у форматних рядках бібліотечних функцій `printf()` і `scanf()` використовується специфікація перетворення `%s`.

Крім `scanf()` і `printf()`, для введення і виведення символів у бібліотеці (заголовний файл `stdio.h`) передбачені спеціальні функції обміну:

`char` `*gets(`**`char`** `*Str);` – зчитує із стандартного вхідного потоку рядок до першого символу нового рядка `'\n'` (включаючи цей символ) і записує зчитаний рядок у символьний масив Str із заміною символу `'\n'` символом `'\0'` (зараз ця функція заблокована);

`int` `puts(`**`const char`** `*Str);` – виводить рядок Str у стандартний вихідний потік (зазвичай на екран дисплея).

Порівняно із `gets()` більш безпечною є функція

`char` `*gets_s(`**`char`** `*Str, size_t` sizeInCharacters`);`

другий параметр якої задає кількість символів у рядку Str.

1.6. Копіювання і зчеплення рядків

`char` `*strcpy(`**`char`** `*strDest,` **`const char`** `*strSource);` – копіює символи рядка strSource в рядок strDest;

`char` `*strncpy(`**`char`** `*strDest,` **`const char`** `*strSource,` **`int`** Count`);` – копіює Count символів рядка strSource в рядок strDest («хвіст» відкидається або доповнюється пробілами);

`char` `*strdup(`**`const char`** `*strSource);` – виділяє пам'ять і переносить у неї копію рядка strSource;

`char` `*strcat(`**`char`** `*strDest,` **`const char`** `*strSource);` – приписує рядок strSource праворуч до рядка strDest (конкатенація рядків);

`char` `*strncat(`**`char`** `*strDest,` **`const char`** `*strSource,` **`int`** Count`);` – приписує Count символів рядка strSource праворуч до рядка strDest (конкатенація).

Функція `strdup()` є застарілою. Замість неї стандарт ISO C++ рекомендує використовувати функцію

`int` `_strdup(`**`const char`** `*strSource);`

Функція повертає нуль у разі успішного виконання; в іншому випадку - помилку.

Оскільки `strcpy()`, `strncpy()`, `strcat()`, `strncat()` не перевіряють наявність достатнього місця в приймаючому рядку strDest перед копі-

юванням strSource, ці функції є потенційною причиною переповнення області пам'яті, у яку йде копіювання. Тому рекомендується використовувати

`errno_t strcpy_s(char *strDest, size_t numberOfElements, const char *strSource);` – аналог функції `strcpy()` із забезпеченням контролю переповнення області пам'яті, у яку здійснюється копіювання; параметр numberOfElements задає розмір цієї області (кількість символів);

`errno_t strncpy_s(char *strDest, size_t numberOfElements, const char *strSource, int Count);` – аналог функції `strncpy()` із забезпеченням контролю переповнення області пам'яті, у яку здійснюється копіювання; параметр numberOfElements задає розмір цієї області (кількість символів); якщо при цьому не буде скопійований завершу вальний символ рядка strDest, то цей символ записується за останнім скопійованим символом;

`errno_t strcat_s(char *strDest, size_t numberOfElements, const char *strSource);` – аналог функції `strcat()` із забезпеченням контролю переповнення області пам'яті, у яку здійснюється копіювання;

`errno_t strncat_s(char *strDest, size_t numberOfElements, const char *strSource, int Count);` – аналог функції `strncat()` із забезпеченням контролю переповнення області пам'яті, у яку здійснюється копіювання; при цьому параметр numberOfElements задає повний розмір пам'яті, відведеної під рядок strDest, а не той, що залишився вільним.

1.7. Пошук у рядках

`char *strchr(const char *Str, int Ch);` – шукає в рядку Str перше входження символу Ch;

`char *strrchr(const char *Str, int Ch);` – шукає в рядку Str останнє входження символу Ch;

`char *strpbrk(const char *Str, const char *strCharSet);` – шукає в рядку Str першу появу будь-якого символу з множини символів, що входять у рядок strCharSet;

`char *strstr(const char *Str, const char *strSearch);` – шукає в рядку Str підрядок strSearch. Повертає покажчик на той елемент у рядку Str, з якого починається підрядок strSearch;

`char *strtok(char *strToken, const char *strDelimit);` – шукає в рядку strToken лексеми, виділені символами-обмежувачами з рядка strDelimit, повертаючи покажчик на перший символ знайденої лексеми; перший виклик цієї функції забезпечує пошук першої лексеми, а для пошуку наступних лексем функцію треба викликати, задаваючи як перший параметр константу `NULL`;

int strcspn(**const char** *Str, **const char** *strCharSet); – визначає індекс першого символу рядка Str, що входить у множину символів рядка strCharSet;

int strspn(**const char** *Str, **const char** *strCharSet); – визначає індекс першого символу рядка Str, що не входить у множину символів рядка *strCharSet, тобто обчислює довжину першого сегмента рядка Str, який містить тільки символи з множини символів рядка strCharSet і починається з першого символу рядка Str.

Порівняно із strtok() більш безпечною є функція

```
char *strtok_s(char *strToken, const char *strDelimit,
char **Next);
```

Третій параметр цієї функції використовується для збереження даних про положення покажчика після відшуканої лексеми.

1.8. Порівняння рядків

int strcmp(**const char** *Str1, **const char** *Str2); – порівнює рядки Str1 і Str2. Результат від'ємний, якщо Str1 < Str2; дорівнює нулю, якщо Str1 == Str2, і додатний, якщо Str1 > Str2 (порівняння беззнакове);

int strncmp(**const char** *Str1, **const char** *Str2, **int** Count); – порівнює частини рядків Str1 і Str2, причому розглядаються перші Count символів. Результат від'ємний, якщо Str1 < Str2; дорівнює нулю, якщо Str1 == Str2, і додатний, якщо Str1 > Str2;

int stricmp(**char** *Str1, **const char** *Str2); – порівнює рядки Str1 і Str2, не роблячи розбіжності регістрів (див. функцію strcmp());

int strnicmp(**char** *Str1, **const char** *Str2, **int** Count); – порівнює не більше Count символів рядка Str1 і рядка Str2, не роблячи розбіжності регістрів (див. функцію strncmp());

Функція stricmp() і strnicmp() застарілим. Замість них стандарт ISO C++ рекомендує використовувати функції

```
int _stricmp(char *Str1, const char *Str2);
int _strnicmp(char *Str1, const char *Str2, int Count);
```

1.9. Трансформування рядків

char *strset(**char** *Str, **int** Ch); – заповнює рядок Str заданим символом Ch;

char *strnset(**char** *Str, **int** Ch, **int** Count); – замінює перші Count символів рядка Str символом Ch;

char *strlwr(**char** *Str); – перетворює літери верхнього регістру в рядку Str у відповідні літери нижнього регістру;

char *strupr(**char** *Str); – перетворює літери нижнього регістру в рядку Str на літери верхнього регістру.

Перелічені вище у цьому підрозділі функції є застарілими. Замість них стандарт ISO C++ рекомендує використовувати функції

```
char *_strset(char *Str, int Ch);
char *_strnset(char *Str, int Ch, int Count);
char *_strlwr(char *Str);
char *_strupr(char *Str);
```

Але більш безпечними є такі функції:

```
errno_t _strset_s(char *Str, size_t numberOfElements, int Ch);
errno_t _strnset_s(char *Str, size_t numberOfElements, int Ch, int Count);
errno_t _strlwr_s(char *Str, size_t numberOfElements);
errno_t _strupr_s(char *Str, size_t numberOfElements);
```

Другим параметром (numberOfElements) цих чотирьох функцій є розмір області пам'яті, що відводиться під рядок Str.

1.10. Перетворення рядкових даних до числових і числових даних до рядкового подання

double atof(**const char** *Str); – перетворює рядок Str у дійсне число типу **double**;

int atoi(**const char** *Str); – перетворює рядок Str у ціле число типу **int**;

long atol(**const char** *Str); – перетворює рядок Str у ціле число типу **long**;

long strtol(**const char** *Str, **char** **endPtr, **int** Baze); – перетворює символьний рядок Str до значення «довге ціле» з основою Baze (2 < Baze < 36). Якщо endPtr не дорівнює NULL, то *endptr повертається як покажчик на символ, при досягненні якого припинено читання рядка Str;

double strtod(**const char** *Str, **char** **endPtr); – перетворює символьний рядок Str до числа подвійної точності. Якщо endPtr не дорівнює NULL, то *endptr повертається як покажчик на символ, при досягненні якого припинено читання рядка Str;

char *itoa(**int** V, **char** *Str, **int** Baze); – перетворює ціле значення V до рядкового подання і записує результат у рядок Str. При зображенні числа використовується основа Baze (2 < Baze < 36). Для від'ємного числа і Baze == 10 першим символом є символ «мінус» (–);

char *ltoa(**long** V, **char** *Str, **int** Baze); – перетворює довге ціле V до рядка Str. При зображенні числа використовується основа Baze (2 < Baze < 36);

char *ultoa(**unsigned long** V, **char** *Str, **int** Baze); – перетворює беззнакове довге ціле V до рядка Str;

char *gcvt(**double** V, **int** Digits, **char** *Str); – перетворює дійсне число V до рядкового подання, що містить Digits значущих цифр, і записує його в рядок Str. Після десяткової крапки у поданні числа не можуть міститися нулі. Якщо число не може бути записане у звичайному десятковому вигляді, воно подається в експоненційному форматі. Обсяг пам'яті, що відводиться для рядка Str, повинен бути достатнім для записування не тільки цифр, але й можливих знаку числа, десяткової крапки, а також ступеня при експоненційному поданні (розміру символьного масиву в _CVTBUFSIZE символів достатньо для запису будь-якого дійсного числа).

Функції itoa(), ltoa(), ultoa(), gcvt() є застарілими. Замість них стандарт ISO C++ рекомендує використовувати функції

```
char *_itoa(int V, char *Str, int Baze);
char *_ltoa(int V, char *Str, int Baze);
char *_ultoa(int V, char *Str, int Baze);
char *_gcvt(double V, int Digits, char *Str);
```

Але більш безпечними є такі функції:

```
errno_t _itoa_s(int V, char *Str, size_t sizeInCharacters,
int Baze);
errno_t _ltoa_s(int V, char *Str, size_t sizeInCharacters,
int Baze);
errno_t _ultoa_s(int V, char *Str, size_t
sizeInCharacters, int Baze);
```

```
errno_t _gcvt_s(char *Str, size_t sizeInBytes, double
V, int Digits);
```

Третім параметром (sizeInCharacters) функцій `_itoa_s()`, `_ltoa_s()`, і `_ltoa_s()` є розмір області пам'яті, що відводиться під рядок Str, а другий параметр функції `_gcvt_s()` задає кількість байт, що відводиться під результуючий рядок Str.

Функції, перелічені в підрозділах 1.6–1.9, описані в заголовному файлі `string.h`, а функції з підрозділу 1.10 – у заголовному файлі `stlib.h`.

Якщо у програмі використовуються застарілі або небезпечні функції, компілятор виводить попередження (warnings).

Щоб заблокувати виведення попереджень про небезпечні функції, необхідно визначити ім'я `_CRT_SECURE_NO_WARNINGS` (це бажано зробити у перших рядках коду):

```
#define _CRT_SECURE_NO_WARNINGS
```

Заборона виведення попереджень про застарілі (нестандартні) функції забезпечується визначенням імені `_CRT_NONSTDC_NO_WARNINGS`:

```
#define _CRT_NONSTDC_NO_WARNINGS
```

2. ПРИКЛАДИ ПРОГРАМ

Приклад 1. Дано рядок із назвою вулиці і два цілих числа, які задають номер будинку та номер квартири. Сформувати рядок із адресою: вулиця, будинок, квартира.

Розв'язок.

```
#include <iostream>
#include <conio.h>
using namespace std;
int main()
{
    char street[256], adress[256]="" , house[5], apartament[5];
    int h, a;
    cout<<"Enter the name of street\n";
    gets(street);
    cout << "Enter the number of house\n";
    cin >> h;
    cout<<"Enter the number of apartament\n";
    cin >> a;
    strcat(strcat(strcat(strcat(strcpy(adress, street),
            ", house "), _itoa(h, house, 10)),", apartament "),
            _itoa(a, apartament, 10));
```

```
    cout << adress;
    cout << "\nPress any key to exit.\n";
    _getch();
    return 0;
}
```

Приклад 2. Дано рядок *s* і символ *ch*. Видалити в рядку *s* підрядок, починаючи з першого і закінчуючи останнім входженням символу *ch*.

Розв'язок.

```
#include <iostream>
#include <conio.h>
using namespace std;
int  main()
{
    char s[256], ch, *pt;
    int n;
    puts("Input string:  ");
    gets(s);
    puts("Input symbol:  ");
    ch = getchar();
    pt = strchr(s, ch);           // Покажчик на перше входження
    if (pt != NULL)                          // Символ виявлений?
    {
        n = strrchr(s, ch) - pt + 1;      // Кількість символів,
                                              // що видаляються
        for (pt = strrchr(s, ch) + 1;
             *(pt - 1) != '\x0'; pt++)           // Зсув ліворуч
            *(pt - n) = *pt;     // разом із ознакою кінця рядка
        puts(s);
    }
    else
        puts("Delete does not work!");
    cout << "\nPress any key to exit.\n";
    _getch();
}
```

Приклад 3. Дано натуральне число *k* і рядки s_1, s_2 довжиною не більше 255 символів. Здійснити вставку рядка s_2 у позицію *k* рядка s_1.

Розв'язок.

```
#include <iostream>
#include <conio.h>
using namespace std;
int main()
{
    char s1[256], s2[256], ps[256];
```

```
    int k;
    puts("Enter the first string");
    gets(s1);
    puts("Enter the second string");
    gets(s2);
    puts("Enter the number of position");
    scanf("%d", &k);
    strcpy(ps, &s1[k - 1]);
    strcpy(&s1[k - 1], s2);
    puts(strcat(s1, ps));
    cout << "\nPress any key to exit.\n";
    _getch();
    return 0;
}
```

Приклад 4. Дано рядки s_1, s_2 і натуральне число k. Вставити рядок s_2 в позицію k рядка s_1. Визначити функцію вставки.

Розв'язок.

```
#include <iostream>
#include <conio.h>
using namespace std;
                  // Функція повертає покажчик на char (на перший
                                  // символ рядка або значення NULL
char * insert(char *st1, char *st2, int k, int SizeOfSt1);

int main ()
{
   char st1[256], st2[256], *ps;
   int k;
   puts("Enter the first string");
   gets(st1);
   puts("Enter the second string");
   gets(st2);
   puts("Enter the number of position");
   cin >> k;
   ps = insert(st1, st2, k, sizeof(st1));
   if (ps == NULL)
     cout<<"Error. An insertion did not come true\n";
   else
     cout<< ps << '\n';
   cout << "Press any key to exit.\n";
   _getch();
   return 0;
}

char * insert(char *st1, char *st2, int k, int SizeOfSt1)
```

```
{
    char *tmp;
    int n;
            // Якщо позиція вставки виходить за межі відведеної
                    // пам'яті, функція повертає значення NULL
    if ((k < 1) || (k > (SizeOfSt1 - 1)))
        return NULL;
    k = k > strlen(st1) ? strlen(st1) + 1 : k;           // Якщо
                  // позиція вставки більше фактичної довжини
                      // рядка-приймача,то готуємо вставку за
                                          // останнім символом st1
    tmp = strdup(&st1[k - 1]);             // Виділяємо пам'ять і
                      // переносимо в неї підрядок рядка st1,
                                    // починаючи з позиції вставки
          // Обчислюємо обсяг пам'яті, що залишився для вставки
    n = SizeOfSt1 - k + 1;
    strncpy(&st1[k - 1], st2, n);
    st1[k + n - 2] = '\x0';
                // Копіюємо не більше n символів із рядка st2
                  // у рядок st1, починаючи з позиції (k - 1)
                            // і приєднуємо ознаку кінця рядка
                                          // Так буде невірно:
              // strcat(strncpy(&st1[k - 1], st2, n), "\x0");
    n = SizeOfSt1 - strlen(st1) - 1;
    return (n > 0 ? strncat(st1, tmp, n) : st1);   // Приєднання
                           // не більше n символів до рядка st1
}
```

Приклад 5. Видалити в рядку *k* символів, починаючи з позиції *n*. Визначити функцію видалення з рядка його частини.

Розв'язок.

```
#include <iostream>
#include <conio.h>

using namespace std;
char * delstr(char *st1, int n, int k);      // Функція повертає
        // покажчик на char (на перший символ) або значення NULL
int main()
{
    char st[256];
    int n, k;
    cout << "Input string\n";
    cin.getline(st, 256);
    cout << "Enter the position number and quantity of"
            " characters to delete: ";
    cin >> k >> n;
```

```
    if (delstr(st, k, n) == NULL)         // st - покажчик. Тому
                // фактичним параметром є не &st, а ім'я рядка st
        cout << "Incorrect parameter in the function!";
    else
        cout << st;
    cout << "\nPress any key to exit.\n";
    _getch();
}
char * delstr(char *st1, int k, int n)
{
    if ((k < 1) || (k > strlen(st1)) || (n < 0))
        return NULL;                  // Якщо здійснюється спроба
                    // видалення поза рядка або спроба видалення
                         // від'ємної кількості символів, st1 не
                  // змінюється, а функція повертає значення NULL
    if (strlen(st1) - k + 1 < n )
        n = strlen(st1) - k + 1;           // Якщо пропонується
                     // видалити символи, кількість яких більше,
                  // ніж їх є до кінця рядка, змінюємо k так, щоб
               // забезпечити видалення до кінця рядка. При цьому
                    // зміненні k невідомо в точці виклику функції
         // Нижче здійснюється видалення за допомогою копіювання
    strcpy(&st1[k - 1], &st1[k - 1 + n]);
    return st1;          // При нормальному завершенні повертаємо
                                                  // покажчик st1
}
```

Приклад 6. Дано рядок *s*. Якщо він перетворюється до цілого числа, отримати це число, указавши основу системи числення, у якій воно було подано в рядку *s*. Розглянути всі можливі в С++ системи числення.

Розв'язок.

```
#include <iostream>
#include <conio.h>
using namespace std;
int main()
{
    char s[256], *endptr;
    puts("Enter the string:\n");
    gets(s);
    for (int radix = 2; radix <= 36; radix++)
    {
        long n  = strtol(s, &endptr, radix);
        if (*endptr == '\x0')
            printf("%ld == %s(%d)\n", n, s, radix);
    }
```

```
    cout << "\nPress any key to exit.\n";
    _getch();
}
```

3. ВАРІАНТИ ЗАДАЧ

1. Дано рядок. Визначити кількість слів[1], що містяться в ньому.
2. Дано рядки символів *s* і *g*. Скільки разів рядок *g* зустрічається в рядку *s*?
3. Дано рядок *s* і символ *c*. Визначити кількість слів, що містяться в ньому. Розглядати тільки такі слова, що:
   а) починаються із символу *c*;
   б) закінчуються символом *c*;
   в) містять хоча б один символ *c*;
   г) не містять символ *c*;
   д) мають рівно три входження символу *c*.
4. Дано рядок. Визначити довжину найкоротшого із слів, що містяться в ньому.
5. Дано рядок, що складається з латинських літер (великих і малих) та пробілів. Чи правда, що він (без урахування пробілів) однаково читається зліва направо і справа наліво, тобто є паліндромом?
6. Дано рядок. Визначити кількість слів, що містяться в ньому і є паліндромами.
7. Дано рядок *s*, що містить *n* символів $s_1, s_2, \ldots, s_n$. Якщо цей рядок є паліндромом, тобто $s_1 = s_n$, $s_2 = s_{n-1}, \ldots$, то залишити його без зміни, інакше доповнити цей рядок справа його «дзеркальним» відбиттям без повторення останнього символу (s_1, s_2, …, s_{n-1}, s_n, s_{n-1}, …, s_2, s_1).
8. Дано рядок *s*, символ *c* і натуральне число *n*. Перетворити рядок таким чином, щоб кожне входження символу *c* було замінене на його *n*-разове входження.
9. Дано рядок тексту, що складається зі слів, розділених одним зі знаків: &, #, $, *, –, +, =. Якщо кількість слів у цьому рядку непарна, видалити в ньому центральне слово, у противному випадку поміняти місцями два центральних слова.
10. Дано рядок. Видалити в ньому найдовший підрядок, що складається тільки з цифр. Якщо таких підрядків декілька, то:
    а) видалити перший з них;
    б) видалити останній з них.

---

[1] У загальному випадку під терміном «слово» будемо розуміти будь-яку послідовність символів, що не містить узагальнених пробільних символів і символів керування, тобто символів з кодами, меншими за 33.

11. Дано рядок, що містить одне або декілька слів. Не змінюючи їх місце розташування всередині рядка, розвернути кожне зі слів таким чином, щоб його символи йшли у зворотному порядку відносно початкового порядку проходження.
12. Дано рядки s і g та невід'ємні цілі числа n_1 і n_2. Сформувати новий рядок, зчепивши n_1 перших символів рядка s і n_2 останніх символів рядка g. Якщо значення n_1 (або n_2) перевищує довжину рядка s (g), то в зчепленні повинні брати участь тільки символи, що входять у рядок s (g).
13. Дано рядки символів s і g. Приєднати рядок g праворуч до рядка s. Якщо початок рядка g збігається з кінцем рядка s, при приєднанні рядків виключити таке накладання символів, видаливши найбільшу їх кількість.

    Наприклад:

    s: C++ is one of progr

    g: rogramming languages.

    Результуючий рядок: C++ is one of programming languages.

    Тут видалений підрядок rogr, хоч можна було видалити тільки r.
14. Дано рядки символів s і g. Чи можна з символів, що містяться в рядку s, сформувати рядок g?

    а) Кожен із символів рядка може бути використаний необмежену кількість разів.

    б) Символ, розміщений у будь-якій з позицій рядка s, може бути використаний тільки один раз.
15. Дано рядки символів s і g.

    а) Які символи одночасно входять в обидва ці рядки?

    б) Які символи входять хоча б в один із рядків s і g?

    в) Які символи входять у рядок s, але не входять у g?

    г) Які символи входять у g, але не входять у рядок s?
16. Дано рядок, що містить десяткові цифри. Замінити кожну цифру її назвою англійською мовою, розділяючи кожне зі слів одним пробілом (0 – zero, 1 – one, 2 – two, 3 – three, 4 – four, 5 – five, 6 – six, 7 – seven, 8 – eight, 9 – nine).
17. Рядок складається зі слів (ланцюжків символів, відділених один від іншого одним або декількома пробілами). Вибрати ті зі слів, які є записом цілих чисел, а ті, що залишилися, розсортувати на слова, які являють собою запис дійсних чисел, і слова, що не є записом чисел мовою C++. Сформувати відповідно три рядки, слова в яких розділені одним пробілом.
18. Дано рядок, що містить одне або декілька слів, розділених одним пробілом. Розташувати слова:

а) у порядку зменшення їх довжини;
б) у лексикографічному (абетковому) порядку.

19. Дано рядок, що містить запис десяткового цілого числа. Одержати суму цифр цього числа.